

# 다중카메라 환경 차량 패턴 분석 및 인공지능을 활용한 차량 추적 시스템

2025년 12월

안양대학교 창의융합대학

컴퓨터공학과

이 준 호

지 도 교 수 하 성 민

이 논문을 공학 학사 학위논문으로 제출함.

2025년 12월

안양대학교 창의융합대학

컴퓨터공학과

이 준 호

# 논문인준서

이 논문을 이준호의 공학 학사 학위논문으로 인준함.

2025년 11월

주 심 김 성 규 인

부 심 하 은 용 인

부 심 하 성 민 인

# 차 례

|                                                      |    |
|------------------------------------------------------|----|
| 요약 .....                                             | 1  |
| 제 1장 서론 .....                                        | 1  |
| 제 1절 연구 배경 .....                                     | 1  |
| 제 2절 연구 필요성 및 기대효과 .....                             | 2  |
| 제 3절 논문의 구성 .....                                    | 3  |
| 제 2장 관련 연구 .....                                     | 4  |
| 제 1절 객체 탐지(Object Detection) 기술 .....                | 4  |
| 제 2절 단일 카메라 다중 객체 추적(MOT) 기법 .....                   | 7  |
| 제 3절 차량 재식별(Re-ID) 기술 .....                          | 9  |
| 제 4절 다중 카메라 기반 차량 추적(Multi-Camera Tracking) 연구 ..... | 11 |
| 제 3장 시스템 설계 .....                                    | 13 |
| 제 1절 전체 시스템 구조 .....                                 | 13 |
| 제 2절 스트림 관리 모듈(Stream Manager) 설계 .....              | 16 |
| 제 3절 프레임 전처리 및 ROI 설계 .....                          | 18 |
| 제 4절 인공지능 기반 차량 탐지 모듈 설계 .....                       | 21 |
| 제 5절 단일 카메라 기반 차량 추적 모듈 설계 .....                     | 23 |
| 제 6절 색상 히스토그램 기반 차량 Re-ID 설계 .....                   | 24 |
| 제 7절 다중 카메라 핸드오버 설계 .....                            | 26 |
| 제 4장 시스템 구현 .....                                    | 28 |
| 제 1절 시스템 구현 환경 .....                                 | 28 |
| 제 2절 전체 파이프라인 구성 요약 .....                            | 29 |
| 제 3절 스트림 입력 및 전처리 구현 .....                           | 30 |
| 제 4절 차량 탐지·추적 및 Re-ID 구현 .....                       | 31 |
| 제 5절 다중 카메라 핸드오버 및 UI 구현 .....                       | 32 |

|                       |    |
|-----------------------|----|
| 제 6절 시스템 실행 화면 .....  | 34 |
| 제 5장 실험 및 성능 평가 ..... | 35 |
| 제 1절 실험 환경 구성 .....   | 35 |
| 제 2절 실험 결과 .....      | 36 |
| 제 3절 한계 및 고찰 .....    | 41 |
| 제 6장 결론 및 향후 과제 ..... | 46 |
| 제 1절 연구 결과 요약 .....   | 46 |
| 제 2절 연구의 한계 .....     | 47 |
| 제 3절 향후 연구 과제 .....   | 48 |
| [참고 문헌] .....         | 49 |

# 표 및 그림 차례

## 표 차례

|                                  |    |
|----------------------------------|----|
| [표 1] 개발 환경 .....                | 28 |
| [표 2] 엔트리 흐름 및 모듈 호출 관계 요약 ..... | 29 |
| [표 3] 스트림 입력 및 전처리 구성 요약 .....   | 30 |
| [표 4] 탐지·추적·Re-ID 구성 요약 .....    | 31 |
| [표 5] 핸드오버 및 UI 구성 요약 .....      | 33 |
| [표 6] 실험 환경 .....                | 35 |
| [표 7] 신뢰도 구간표 .....              | 40 |
| [표 8] 시스템 구조 변화 및 설계 비교 .....    | 45 |

## 그림 차례

|                                                            |    |
|------------------------------------------------------------|----|
| [그림 1] YOLOv11과 EfficientDet 간 객체 탐지 성능 비교 그래프. ....       | 6  |
| [그림 2] 주요 다중 객체 추적 알고리즘의 성능 비교 그래프. ....                   | 8  |
| [그림 3] 다중 카메라 기반 차량 탐지·추적 시스템 파이프라인 구성도. ....              | 15 |
| [그림 4] 스트림 관리 모듈의 URL 갱신 및 재연결 흐름도. ....                   | 17 |
| [그림 5] 고속도로 CCTV 영상에서의 ROI(Region of Interest) 지정 예시. .... | 20 |
| [그림 6] 색상 기반 차량 Re-ID를 위한 밴드 분할 및 히스토그램 특징 추출<br>과정. ....  | 25 |
| [그림 7] 고속도로 CCTV 간 물리적 연결 관계를 나타낸 CCTV Graph 구조            | 27 |
| [그림 8] 실행화면 1 .....                                        | 34 |
| [그림 9] 실행화면 2 .....                                        | 34 |
| [그림 10] 실시간 스트리밍 및 사용자 UI 구성 .....                         | 34 |
| [그림 11] ROI(Region of Interest) 적용 유무에 따른 탐지 결과 비교 .....   | 37 |
| [그림 12] 다중 카메라 간 동일 차량 추적 과정. ....                         | 38 |
| [그림 13] 차량 매칭 로그 및 로그 기반 UI 예시. ....                       | 39 |



## 요 약

본 논문에서는 고속도로 구간에 설치된 다수의 교통관제용 CCTV를 활용하여 차량을 실시간으로 탐지하고, 카메라 간 이동 시에도 동일 차량을 연속적으로 추적할 수 있는 인공지능 기반 다중 카메라 차량 추적 시스템을 구현하였다.

기존 연구와 달리 본 연구에서는 별도의 객체 추적이나 복잡한 딥러닝 기반 Re-ID (Re-Identification) 모델을 구축하지 않고, YOLO (You Only Look Once)가 제공하는 내장 tracking 모듈을 활용하여 단일 카메라 내에서의 ID 유지 성능을 확보하였다.

제안 시스템은 국가교통정보센터 Open API (Open Application Programming Interface)를 통해 제공되는 실제 고속도로 CCTV HLS (HTTP Live Streaming) 스트림에 적용하여 평가하였다. 정성적 실험을 통해 ROI (Region of Interest) 기반 탐지, YOLO 기반 단일 카메라 추적, 그리고 다중 CCTV 간 자연스러운 핸드오버가 가능함을 확인하였다.



# 제 1장 서 론

## 제 1절 연구 배경

최근 도심 및 고속도로에서의 교통량 증가에 따라 교통 혼잡 완화와 사고 대응을 위한 지능형 교통 관제 시스템의 중요성이 점차 확대되고 있다. 특히 고속도로 구간에는 다수의 CCTV가 연속적으로 설치되어 있어, 관제 인력은 여러 화면을 동시에 모니터링하며 사고, 정체, 과속 등 다양한 이상 상황을 실시간으로 파악해야 한다. 그러나 이러한 작업은 관제자의 집중력과 숙련도에 크게 의존하기 때문에, 관제 효율과 정확성을 향상시키기 위한 영상 기반 차량 탐지 및 추적 기술의 필요성이 지속적으로 증가하고 있다.

기존 교통 영상 분석 연구는 주로 단일 카메라 환경에서의 객체 탐지와 추적에 초점을 두는 경향이 있었다. 그러나 실제 도로 환경에서는 차량이 여러 대의 카메라를 순차적으로 지나기 때문에, 서로 다른 시점에서도 동일 차량을 안정적으로 식별할 수 있는 다중 카메라 기반 추적 기술이 필수적이다. 이를 해결하기 위해 딥러닝 기반 차량 Re-ID (Re-Identification) 모델을 활용하는 다양한 접근이 제안되어 왔으나, 고해상도 입력 데이터와 복잡한 네트워크 구조로 인해 연산량이 매우 크다. 이로 인해 일반적인 PC 환경이나 엣지 디바이스에서는 실시간 처리 성능을 확보하는 데 한계가 존재한다. 본 연구는 이러한 문제점을 극복하기 위한 실시간 다중 카메라 차량 추적 시스템의 구현을 목표로 한다.

## 제 2절 연구 필요성 및 기대효과

고속도로와 같은 광역 교통 환경에서 차량 이동을 실시간으로 감지하고, 카메라 간 연속적으로 추적하는 기능은 사고 대응, 교통 흐름 분석, 돌발 상황 탐지 등 다양한 분야에서 중요하다. 그러나 상공 시점으로 촬영되는 교통 CCTV의 경우 차량 크기가 작고 해상도가 낮아 고성능 Re-ID 모델이나 대형 탐지 모델을 직접 적용하기 어렵다. 또한 여러 영상 스트림을 동시에 처리해야 하는 실제 관제 환경에서는 연산량 제한까지 존재한다.

본 연구에서는 이러한 제약을 고려하여 실시간성·경량성·소형 객체 검출 성능을 충족할 수 있는 YOLO (You Only Look Once) 계열 모델 중 YOLOv11n을 1차 탐지 모델로 채택하였다. 입력 프레임의 불필요한 영역을 제거하는 Region of Interest (ROI) 기반 입력 축소 기법을 적용하여 연산량을 줄였으며, YOLO 내장 tracking 기능을 이용해 단일 카메라 내에서 ID 유지 기능을 확보하였다. 또한 색상 히스토그램과 Bhattacharyya 거리 기반의 단순 Re-ID 방식으로 복잡한 딥러닝 기반 모델 없이도 다중 카메라 간 차량 연결을 수행할 수 있도록 구현하였다.

기대효과는 다음과 같다.

첫째, 대규모 학습 없이도 실제 고속도로 CCTV 환경에서 다중 카메라 추적이 가능함을 확인하였다.

둘째, ROI 기반 연산량 감소와 경량 모델 활용으로 제한된 하드웨어에서도 안정적인 실시간 처리가 가능하다.

셋째, YOLO tracking과 색상 기반 Re-ID를 결합하여 구현 복잡도를 낮추면서 실용적인 다중 카메라 연계 구조를 제시하였다.

## 제 2절 논문의 구성

본 논문은 총 여섯 개의 장으로 구성된다. 제 1장에서는 연구의 배경, 필요성, 기대효과를 설명하여 본 연구의 수행 목적을 제시한다. 제 2장에서는 객체 탐지, 단일·다중 카메라 추적, 차량 재식별 기법 등 관련 연구와 기술적 배경을 정리한다. 제 3장에서는 본 연구에서 제안한 시스템의 전체 구조, ROI 설계, YOLO 기반 단일 카메라 추적 방식, 그리고 다중 카메라 간 재식별 및 핸드오버 구조를 상세히 기술한다. 제 4장에서는 구현 환경, 데이터 처리 과정, YOLO 기반 추론 파이프라인, 색상 히스토그램 기반 Re-ID 및 전체 시스템의 동작 방식을 구현 관점에서 설명한다. 제 5장에서는 실제 고속도로 CCTV 스트림을 기반으로 수행한 실험 결과를 정성적으로 분석하고, 제안 시스템의 성능과 한계를 논의한다. 마지막으로 제 6장에서는 연구 결과를 요약하고, 향후 개선 방향과 확장 가능성을 제시한다.

## 제 2장 관련 연구

### 제 1절 객체 탐지(Object Detection) 기술

영상 기반 차량 분석 시스템에서 객체 탐지는 전체 파이프라인의 성능을 결정하는 핵심 요소이다. 최근에는 1-stage 구조 기반의 You Only Look Once (YOLO) 계열 모델이 높은 정확도와 빠른 처리 속도를 제공하며, 다양한 교통 영상 분석 분야에서 널리 활용되고 있다[1 - 4]. YOLO 계열 모델은 단일 단계(one-stage) 방식으로 동작하여 높은 FPS를 제공하며, 실시간 교통 관제 시스템이나 CCTV 분석 환경과 같은 저지연 요구 조건에 적합한 것으로 여러 선행 연구에서 보고되고 있다[5].

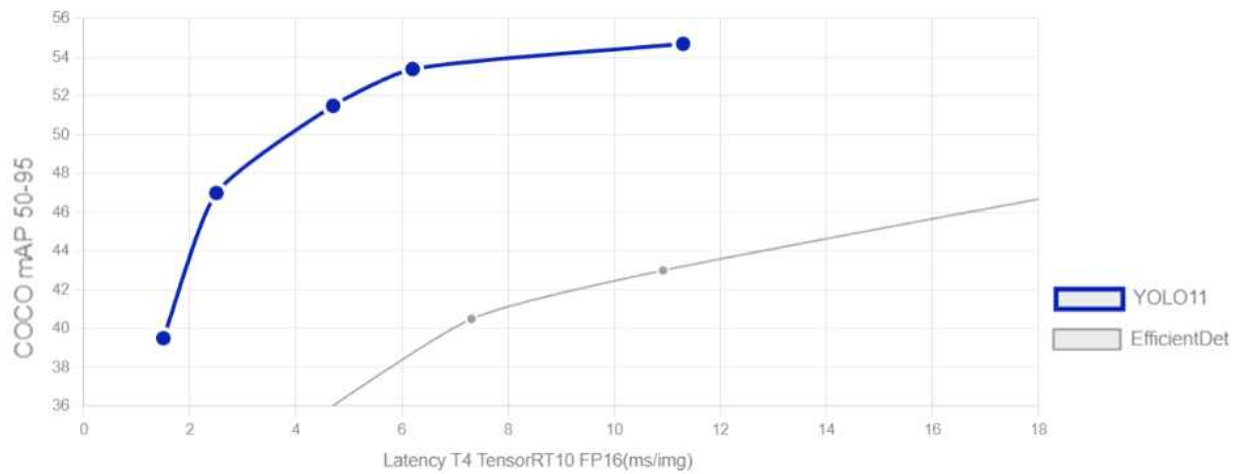
고속도로 CCTV는 넓은 시야각을 확보하기 위해 상공에서 도로를 포착하는 경우가 많으며, 이로 인해 차량이 영상 내에서 매우 작은 크기로 나타나고 배경 비율이 높아지는 문제가 존재한다. 또한 여러 지점의 영상을 동시에 처리해야 하는 실제 관제 환경에서는 탐지 모델이 충분히 경량화되어야 하며, 작은 객체에 강인한 성능을 가져야 한다. 이러한 이유로 차량 객체 탐지에서는 YOLO, SSD, CenterNet 등 다양한 모델이 연구되었으며, 이들 간 성능 비교 연구도 활발히 이루어지고 있다[6]. 국내 연구에서도 YOLOv4, SSD, CenterNet을 대상으로 차량 탐지 성능을 비교하여 YOLO 계열이 실시간 교통 환경에서 우수한 정확도와 효율성을 보인다고 분석한 바 있다[7].

이러한 요구 조건을 고려하여, 본 연구에서는 YOLO 계열 모델과 함께 대표적인 경량 객체 탐지 모델인 EfficientDet을 비교 대상으로 검토하였다. EfficientDet은 Bi-directional Feature Pyramid Network (BiFPN)을 활용해 정확도와 연산량의 균형을 달성한 모델로, 모바일·엣지 환경에서의 활용성이 널리 보고되고 있다[8]. 그러나 EfficientDet-D0~D1 수준의 경량 모델은 입력 해상도

를 충분히 확보하지 않는 경우 작은 객체에 대한 검출 성능이 제한적이며, 원거리에서 촬영된 도로 차량을 빠르게 인식해야 하는 고속도로 CCTV 환경에서는 민감도가 충분하지 않은 것으로 알려져 있다. 또한 EfficientDet은 구조적 특성상 YOLO 계열보다 추론 속도가 느린 경향이 있어 여러 CCTV 영상 스트림을 병렬 처리해야 하는 관제 시스템에서는 처리 지연이 발생할 수 있다는 한계가 지적되고 있다[8].

반면 YOLO 계열은 경량 백본과 단일 단계 구조를 통해 연산량을 최소화하면서도 작은 객체에 대한 감지 성능이 우수하여, 실시간 CCTV 기반 차량 감지 시스템에 적합하다는 평가를 받고 있다[9]. 특히 차량이 화면의 일정 영역에만 존재하는 고속도로 CCTV 특성상 Region of Interest (ROI) 기반 입력 축소 기법을 적용할 수 있으며, YOLO는 이러한 전처리와 결합했을 때 탐지 성능과 속도 모두에서 안정적인 결과를 제공한다.

이와 같은 선행 연구 분석을 바탕으로, 본 연구에서는 고속도로 CCTV 환경의 제약 조건인 작은 객체 검출, 다중 스트림 실시간 처리, 시스템 통합성 등을 종합적으로 고려하여 YOLO 계열 모델을 1차 탐지기로 채택하였다.



[그림 1] YOLOv11과 EfficientDet 간 객체 탐지 성능 비교 그래프.

X축은 NVIDIA T4 GPU에서 TensorRT FP16으로 측정한 이미지당 추론 지연(ms),  
Y축은 COCO 데이터셋 기반 mAP 50 - 95 정확도를 의미한다.

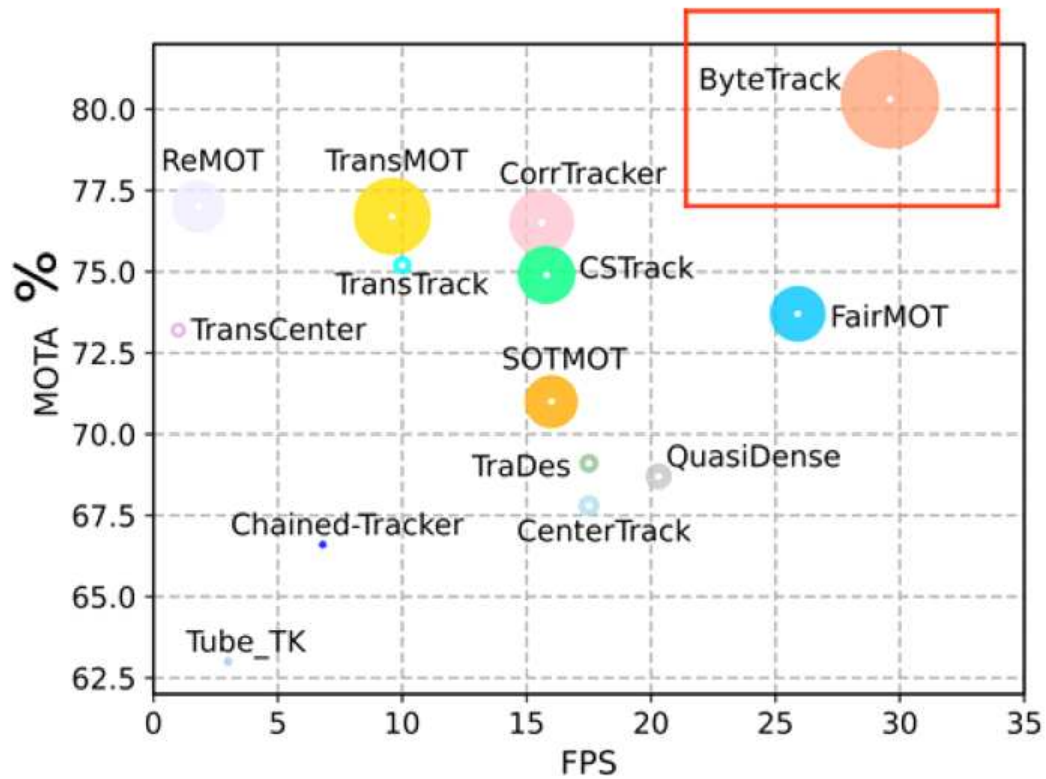
동일한 추론 지연 조건에서 YOLOv11이 EfficientDet보다 더 높은 정확도를 기록하여  
경량·실시간 환경에서의 우수성을 확인할 수 있다.

## 제 2절 단일 카메라 다중 객체 추적(MOT) 기법

다중 객체 추적(Multi-Object Tracking, MOT)은 연속된 영상 프레임에서 동일 객체를 식별하고 추적하는 기술이다. 일반적으로 Kalman filter를 이용한 상태 예측과 Hungarian algorithm을 이용한 탐지 매칭 기법이 결합되어 사용된다 [10,11]. 최근에는 ByteTrack, FairMOT, CenterTrack과 같은 탐지 기반 추적 기법이 제안되었으며[12 - 14], 특히 ByteTrack은 높은 MOTA (Multiple Object Tracking Accuracy) 성능과 빠른 처리 속도로 널리 활용되고 있다. ByteTrack은 검출 신뢰도가 낮은 bounding box까지 활용하는 전략을 통해 공공 데이터셋에서 기존 방식 대비 우수한 추적 정확도를 보여 대표적인 실시간 MOT 알고리즘으로 평가된다[12].

이러한 독립 추적기들은 탐지 단계와 추적 단계를 별도로 구성해야 하며, 각 모듈 간 연동 과정에서 후처리 복잡도가 증가한다는 단점이 있다. 반면 Ultralytics YOLO는 전통적인 MOT 구조를 기반으로 한 내장 tracking module을 제공하여 탐지와 추적을 단일 파이프라인에서 처리할 수 있다는 장점이 있다 [15]. 본 연구에서는 이러한 구조적 단순성과 실시간 처리 요구를 고려하여 별도의 추적기 구성 없이 YOLO 내장 트래커를 이용해 단일 카메라 내 차량 ID를 안정적으로 유지하였다.

고속도로 CCTV는 카메라 시점이 고정되고 조명 변화가 비교적 적으며 차량 이동 방향이 일정해 단일 카메라 기반 추적에 적합한 환경을 제공한다. 그러나 상공에서 촬영된 영상 특성상 차량 크기가 작아 탐지 누락이 발생하기 쉽고, 이러한 누락은 추적 단절로 이어질 수 있다. YOLO 내장 추적기는 이러한 누락을 Kalman filter 기반 상태 예측으로 보완하여 고속도로 CCTV 조건에서 안정적인 추적 성능을 확보할 수 있도록 한다.



[그림 2] 주요 다중 객체 추적 알고리즘의 성능 비교 그래프.

X축은 처리 속도를 의미하는 초당 프레임 수(FPS), Y축은 추적 정확도를 의미하는 Multiple Object Tracking Accuracy (MOTA)(%)를 나타낸다. 각 점은 알고리즘별 성능을 의미하며, 우측 상단에 위치할수록 높은 정확도와 속도를 동시에 달성한 모델임을 의미한다.



### 제 3절 차량 재식별(Re-ID) 기술

차량 재식별(Re-Identification, Re-ID)은 서로 다른 카메라에서 촬영된 차량 이미지들 중 동일 차량을 식별하는 기술로, 다중 카메라 기반 추적 시스템의 핵심 요소로 활용된다. 기존 연구에서는 주로 딥러닝 기반 특징 추출 모델을 사용하여 차량의 외관, 색상, 형태적 패턴을 임베딩 형태로 표현한 뒤, 카메라 간 유사도 비교를 통해 동일 차량 여부를 판별하였다. 대표적으로 Liu et al.(2016)이 구축한 VeRi-776 데이터셋과 CNN 기반 Re-ID 기법은 차량 외관 특징을 학습하여 높은 식별 성능을 보였고[16], 이후 He et al.(2019)과 Shen et al.(2017)은 시점 변화(viewpoint variation)와 조명 변화에 강인한 특성을 학습하는 다양한 구조를 제안하였다[17,18].

그러나 이러한 딥러닝 기반 Re-ID 모델은 충분한 해상도와 다양한 각도에서 촬영된 차량 학습 데이터가 필요하며, 연산량이 높아 실시간 교통 영상 처리 환경에서는 적용이 제한적이다. 특히 고속도로 CCTV는 넓은 구간을 상공에서 촬영하기 때문에 차량이 매우 작은 크기로 나타나며, 차량 외관 정보가 부족해 딥러닝 기반 모델이 학습한 특징을 안정적으로 활용하기 어렵다.

이러한 현실적 제약으로 인해 실제 교통 관제와 같은 실시간 환경에서는 색상 기반 또는 분포 기반의 경량화된 통계적 Re-ID 방식도 함께 연구되고 있다. 특히 HSV 색상 공간은 조명 변화와 색상 정보를 분리하여 표현할 수 있기 때문에, 영상 조건이 불규칙한 CCTV 환경에서도 비교적 안정적인 색상 특성을 유지할 수 있다는 장점이 보고되었다. Tangkawanit & Kanprachar(2020)은 조도 변화가 큰 환경에서 HSV 구성 요소(Hue, Saturation, Value)의 특징이 차량 탐지에 미치는 영향을 분석하여, 특정 배경 조건에서 H 또는 S 채널이 높은 안정성을 보인다고 제안하였다[19]. 또한 국내에서도 HSV 기반의 차량 프레임 분석 및 차량 관리 시스템 연구가 수행되어, HSV 색상 공간이 RGB 대비 더 높은 재

현율과 조명 변화 안정성을 제공한다는 실험 결과가 보고된 바 있다[20]. 더 나아가 패턴 매칭과 HSV 컬러 모델을 결합한 차량 인식 연구에서는 HSV 기반 색상 분석이 그림자·조명 변화에 강인하고 오탐지 감소에 효과적인 것으로 나타나, 실시간 교통 영상 분석에서 색상 기반 기법의 활용 가능성을 시사하였다 [21].

이와 같은 선행 연구들은 고속도로 CCTV와 같이 작은 차량 크기, 조도 변화, 시점 문제 등 딥러닝 기반 Re-ID가 안정적으로 적용되기 어려운 환경에서는 경량 색상 특징 기반 접근이 실용적인 대안이 될 수 있음을 보여준다. 본 연구는 이러한 특성을 반영하여, 차량 외관 임베딩 대신 색상 히스토그램과 Bhattacharyya distance 기반의 통계적 Re-ID 방식을 적용하여 다중 카메라 간 차량 연결을 구현하였다.

## 제 4절 다중 카메라 기반 차량 추적(Multi-Camera Tracking) 연구

다중 카메라 기반 차량 추적(Multi-Camera Tracking, MCT)은 차량이 여러 CCTV 간을 이동할 때 동일 객체임을 판별하여 연속적인 이동 경로를 구성하는 기술이다. 기존 MCT 연구는 크게 두 가지 방식으로 구분된다. 첫째는 딥러닝 기반 Re-ID 방법으로, 차량 외관을 임베딩 벡터(feature embedding)로 표현하고 카메라 간 거리 비교를 통해 동일 차량 여부를 판단하는 방식이다. 예를 들어 Chen et al.(2018)은 딥러닝 기반 특징 벡터를 활용하여 카메라 간 유사도를 측정하는 구조를 제안하였으며[22], 이후 다양한 딥러닝 Re-ID 모델이 제시되어 왔다. 이러한 방식은 고품질 차량 이미지가 확보되는 환경에서는 높은 정확도를 보이지만, 원거리 촬영이 일반적인 교통 CCTV에서는 외관 정보가 부족해 성능이 크게 저하되는 한계가 있다[23].

둘째는 이동 가능 경로 및 도로 구조를 활용하는 방식이다. Tang et al.(2018)은 카메라 간 공간적 연결 관계를 그래프 형태로 구성하여 실제 차량이 이동 가능한 경로만을 후보로 고려하는 방식을 제안하였으며[24], 이는 불필요한 비교를 줄이고 계산 효율을 높인다. 고속도로와 같이 차량 이동 방향이 명확한 환경에서는 속도 제약(speed constraint), 이동 시간 제약(temporal constraint), 진입·이탈 지점 기반 후보 축소와 같은 물리적 조건을 활용하는 연구도 수행되어 왔다[25]. 이러한 방식은 영상 품질이 낮거나 차량 외관 정보가 제한된 경우에도 물리적·공간적 단서를 활용하여 비교 대상 범위를 효과적으로 줄일 수 있다는 장점이 있다.

한편, 고해상도 학습 데이터가 부족한 실제 교통 CCTV 환경에서는 딥러닝 기반 임베딩보다 색상 분포 또는 지역적 특징을 활용한 경량화된 MCT 접근도 연구되고 있다. Zhao et al.(2020)은 색상 히스토그램과 지역적 색상 특징을 활용하여 카메라 간 동일 차량 여부를 판별하는 경량 방식의 MCT 기법을 제안하였으

며[26], 이는 고품질 학습 데이터가 필요하지 않고 연산량이 낮아 실시간 교통 관제 시스템에서 활용도가 높다고 평가된다.

또한 homography transform을 활용하여 카메라 간 위치 정보와 시점 차이를 보정하는 연구도 제안되고 있다. 장준혁 외(2023)는 딥러닝 기반 Re-ID 특징과 homography transform을 결합하여 카메라 간 centroid 거리 기반 유사도 계산을 수행함으로써 외형이 유사한 차량 간 매칭 오류를 줄이고 계산량을 크게 증가시키지 않으면서 성능을 개선하는 방법을 제안하였다[27].

국내 연구에서도 다중 카메라를 활용한 광역 객체 추적 시스템이 연구되어 왔다. 방상성(2015)은 두 대 이상의 고정 카메라를 이용하여 광범위 영역에서 움직이는 객체를 추적하는 알고리즘을 제안하고, 다중 카메라 간 협력 기반 객체 검출·추적 구조의 효용성을 실험적으로 분석하였다[28]. 이러한 연구들은 실제 교통 관제 환경에서 다중 카메라 기반 추적 기술이 연산량, 이미지 품질, 공간적 제약 등을 고려해야 하며, 경량·실시간 방식이 중요한 방향임을 시사한다.

종합하면, 기존 MCT 연구는 딥러닝 기반 외관 특징 비교 방식, 물리적 제약 기반 후보 축소 방식, 색상·통계 기반 경량 방식, homography 기반 위치 보정 방식 등 다양한 접근으로 발전해 왔다. 특히 고속도로 CCTV와 같이 원거리·저해상도 환경에서는 연산 효율과 견고성을 동시에 확보할 수 있는 경량 MCT 방식이 중요한 연구 방향으로 자리 잡고 있다.

## 제 3장 시스템 설계

### 제 1절 전체 시스템 구조

본 연구에서 제안하는 다중 카메라 기반 차량 추적 시스템은 고속도로 CCTV에서 수집되는 실시간 영상을 기반으로 차량을 탐지하고, 단일 카메라 내 추적과 카메라 간 재식별(Re-Identification, Re-ID)을 수행하여 차량 이동 경로를 연속적으로 연결하는 것을 목표로 한다. 이를 위해 영상 입력 단계부터 Re-ID 및 핸드오버 단계까지 일관된 파이프라인을 구성하였으며, 각 모듈은 독립적으로 설계하되 상호 간 데이터 흐름이 자연스럽게 이어지도록 계층적으로 배치하였다.

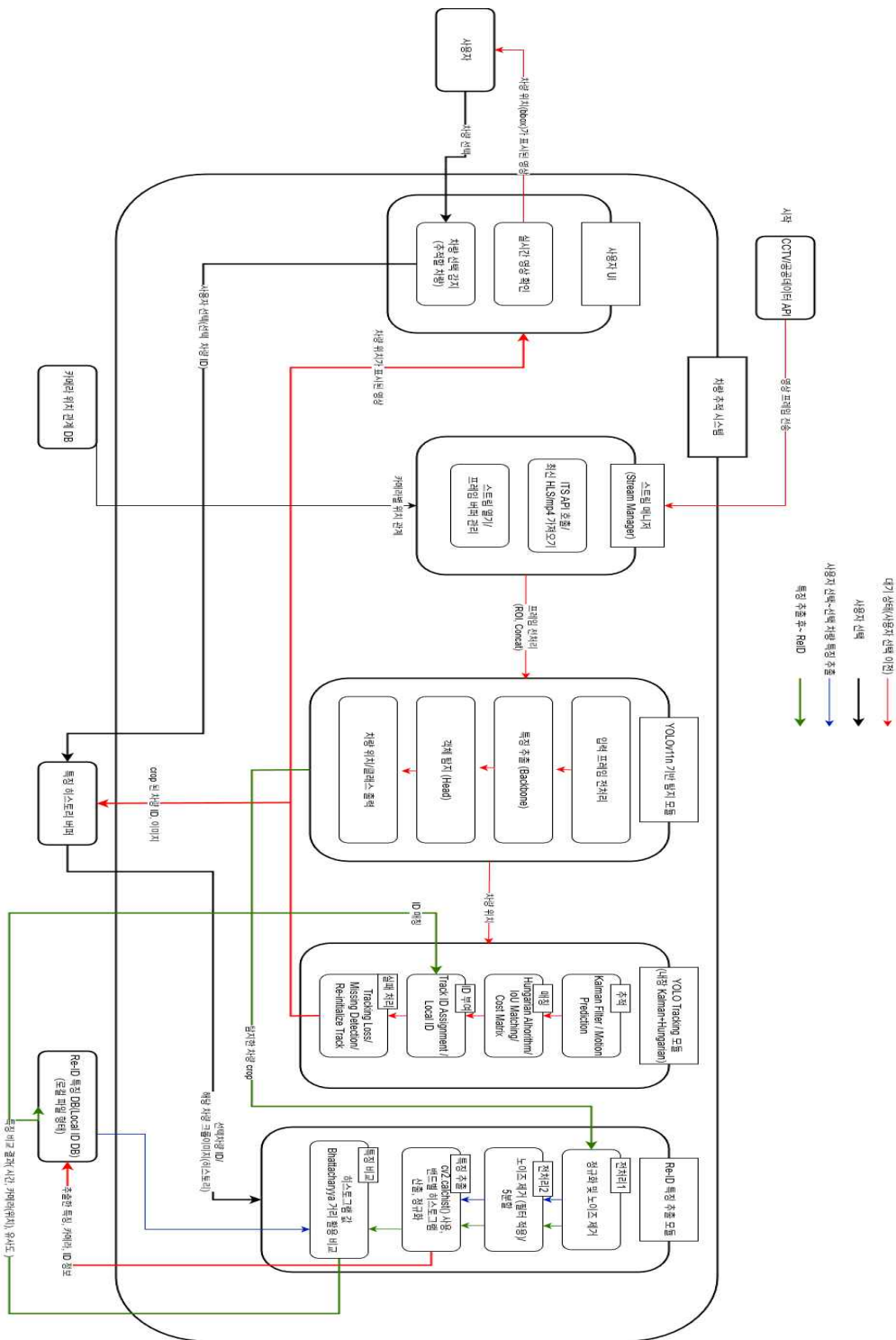
시스템은 크게 스트림 수집 및 관리, 프레임 전처리, YOLO(You Only Look Once) 기반 차량 탐지 및 단일 카메라 추적, 색상 기반 차량 Re-ID, 그리고 다중 카메라 간 핸드오버 모듈로 구성된다.

스트림 관리 모듈(Stream Manager)은 국가교통정보센터 Open API(Application Programming Interface)를 주기적으로 호출하여 각 CCTV의 최신 HLS(HTTP Live Streaming) URL을 갱신하고, 각 스트림의 최신 프레임을 버퍼링한다.

전처리 단계에서는 ROI(Region of Interest) 기반 영상 크롭과 해상도 조정을 통해 불필요한 영역을 제거하여 추론 효율을 높인다. 이후 YOLOv11n 모델이 차량을 탐지하고, YOLO 내장 tracking 모듈이 Kalman filter와 Hungarian algorithm을 활용해 단일 카메라 내 ID를 유지한다.

탐지·추적된 차량 중 후속 카메라로 이동할 가능성이 있는 객체에 대해서는

crop 이미지를 기반으로 HSV(Hue - Saturation - Value) 색상 공간 기반 히스토그램 특징을 추출하고, 카메라 위치 관계 데이터베이스(DB)와 결합하여 후속 카메라에서 동일 차량 여부를 판단한다. 이러한 구조를 통해 실시간 추론 요구사항을 만족시키면서도 카메라 수 증가에 대응 가능한 확장성을 확보하였다.



[그림 3] 다중 카메라 기반 차량 탐지·추적 시스템 파이프라인 구성도.

영상 입력부터 차량 탐지, 단일 카메라 추적, 색상 기반 Re-ID, 후속 카메라 선택 및 핸드오버에 이르는 전체 처리 과정을 모듈 단위로 나타낸다.

## 제 2절 스트림 관리 모듈(Stream Manager) 설계

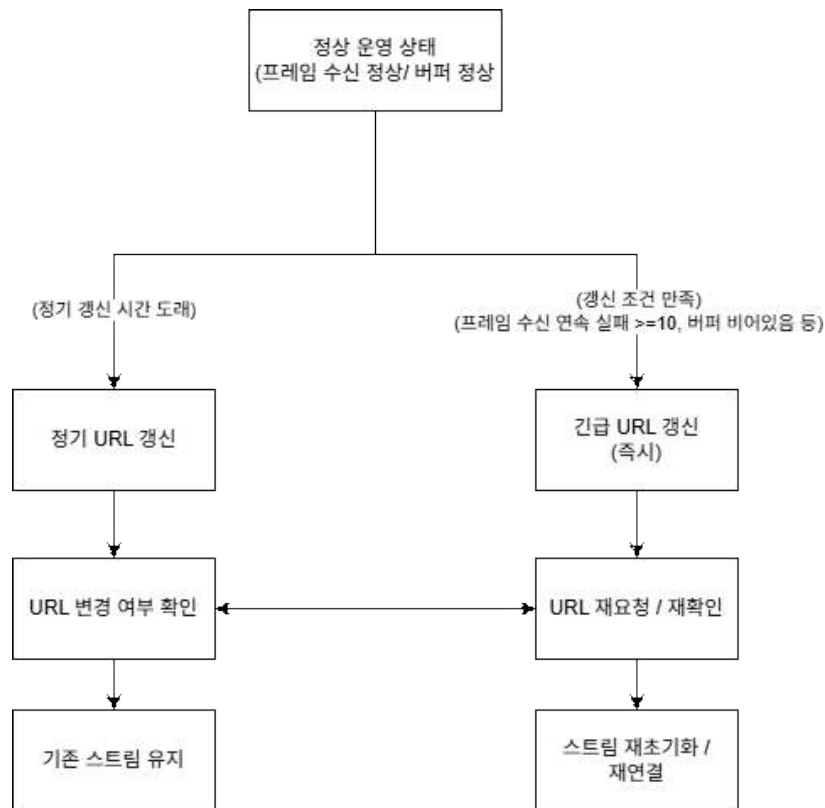
고속도로 CCTV는 HLS 기반 영상 스트림을 제공하며, 네트워크 상황에 따라 프레임 지연, 끊김, URL 갱신 지연 등이 발생할 수 있다. 이러한 환경에서 안정적으로 실시간 영상을 수신하기 위해 본 연구에서는 전용 스트림 관리 모듈(Stream Manager)을 설계하였다. 본 모듈은 영상 입력 단계에서 발생하는 불안정성을 내부적으로 처리하여, 탐지·추적 모듈이 항상 최신 프레임을 활용할 수 있도록 보장한다.

스트림 관리 모듈은 ITS OpenAPI를 주기적으로 호출해 각 CCTV의 최신 HLS/mp4 주소를 수신한다. 고속도로 CCTV 스트림은 일정 주기마다 URL이 갱신되므로 URL을 고정하면 일정 시간이 지난 후 스트림이 끊기거나 재생이 중단되는 문제가 발생한다. 이를 방지하기 위해 스트림 매니저는 일정 주기로 API를 재호출하여 최신 URL을 확인하고, 변경이 감지될 경우 스트림을 자동으로 재초기화하도록 구성하였다. 또한 연속 프레임 수신 실패, 최근 성공 시점 경과, 버퍼 비어 있음과 같은 스트림 품질 저하 신호가 감지될 경우 즉시 URL을 갱신하는 스마트 갱신(smart update) 조건을 함께 적용하여, 스트림 상태 변동에 신속히 대응할 수 있도록 하였다.

스트림 연결 이후에는, 프레임이 과도하게 누적되는 문제를 방지하기 위해 단일 최신 프레임 버퍼 구조를 사용한다. 스트림 매니저는 백그라운드 스레드에서 지속적으로 프레임을 수신하며, 탐지 모듈은 필요 시 스트림 매니저로부터 “가장 최신 프레임”만을 요청하여 처리한다. 이를 통해 네트워크 지연이나 처리 속도 저하로 인해 영상이 뒤처지는 문제를 최소화할 수 있다. 아울러 스트림 실패가 반복될 경우 보조 입력 스트림(backup stream)으로 자동 전환하는 이중 스트림 구조를 적용하여, 일시적인 연결 장애에도 실시간 영상 입력이 지속되도록 안정성을 강화하였다.



또한 일정 시간 동안 프레임이 입력되지 않을 경우, 스트림 매니저는 스트림 재연결을 시도하거나 API를 다시 호출해 새로운 URL을 확보한다. 실패 시에는 설정된 대기 시간을 적용한 후 재시도하도록 하여 네트워크 변동 상황에서도 안정적인 영상 스트림 유지가 가능하도록 하였다. 다중 카메라 환경에서는 각 스트림을 독립적으로 관리하므로, 특정 카메라에서 오류가 발생하더라도 시스템 전체가 중단되지 않고 지속적으로 운영될 수 있다.



[그림 4] 스트림 관리 모듈의 URL 갱신 및 재연결 흐름도.

Stream Manager에서 적용되는 정기 갱신 및 긴급 갱신 기반 스트림 안정화 절차. 스트림 품질 저하 또는 연속 실패 조건이 충족될 경우 즉시 URL 갱신 및 재초기화를 수행하여 실시간성을 유지한다.

### 제 3절 프레임 전처리 및 ROI 설계

고속도로 CCTV는 넓은 구간을 촬영하기 때문에 프레임에서 차량이 차지하는 비율이 매우 작고, 특히 화면 상단부로 갈수록 객체 크기가 급격히 감소한다. 이 구간에서는 픽셀 수가 부족하여 YOLO 탐지의 신뢰도가 떨어지고, 작은 객체를 오탐지하거나 탐지를 놓치는 경우가 증가한다. 이로 인해 tracking 단계에서도 ID 스위칭 및 추적 단절이 발생하는 문제가 확인되었다.

본 연구에서는 이러한 원거리 구간을 입력 단계에서 제거하고, 차량 크기가 충분히 확보되는 중·근거리 영역만을 ROI로 설정하였다. 이는 작은 객체로 인한 탐지 실패를 줄일 뿐 아니라, 배경 요소 제거를 통해 탐지 안정성과 tracking 일관성을 동시에 향상시키는 효과가 있다. 특히 본 연구 환경에서는 차량의 분포가 화면 하단부에 집중되는 특성을 활용하여 ROI를 최적화하였다.

전처리 과정은 카메라마다 정의된 ROI 좌표를 활용해 프레임에서 차로 영역만을 크롭하는 단계로 시작된다. 이를 통해 입력 크기를 줄여 연산량을 감소시키고, 배경으로 인해 발생하는 불필요한 탐지를 제거하였다. ROI 설정 정보는 JSON 파일 형태로 저장되어 스트림 매니저와 탐지 모듈에서 공통적으로 사용되며, 카메라별 해상도와 시야각 차이에 따라 독립적으로 유지된다.

이후 ROI 영역은 YOLO 입력 규격에 맞게 크기 조정(resize)되며, 조명 변화에 민감한 색상 정보를 보완하기 위해 Re-ID 모듈에서는 HSV 색상 공간을 사용한다. 전처리된 프레임은 텐서 형태로 변환된 후 탐지기 입력으로 전달되며, 동일한 처리 흐름을 통해 다중 카메라 환경에서도 일관된 입력 구조가 유지된다.

본 연구의 전처리 구조는 ROI 기반 입력 축소와 표준화된 해상도 조정을 통해 실시간 처리 성능 향상에 기여하였으며, 실험에서도 ROI 적용 시 FPS 증가와

오탐률 감소 효과가 확인되었다. 이를 통해 ROI 설계가 고속도로 CCTV 기반 차량 탐지·추적 환경에서 중요한 역할을 수행함을 검증하였다.

본 연구에서는 전체 프레임 대신 ROI 영역만을 YOLO 모델 입력으로 사용하여 연산량을 효과적으로 절감하였다. 전체 영상의 면적을  $A_{full}$ , ROI 영역의 면적을  $A_{ROI}$ 라 하면 ROI 비율은 다음과 같이 정의된다.

$$R_{ROI} = \frac{A_{ROI}}{A_{full}}$$

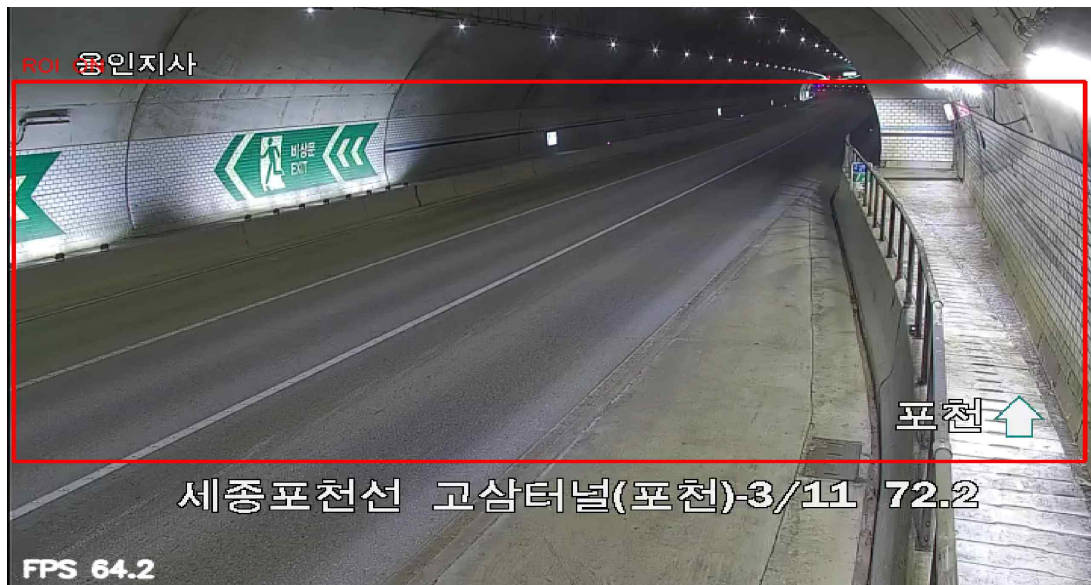
연산량 감소율은 다음과 같다.

$$R_{reduce} = 1 - R_{ROI}$$

본 연구에서 사용된 CCTV의 원본 해상도는  $720 \times 480 = 345,600\text{px}$ 이며, 설정된 ROI 좌표 (4,62,712,375)에 따른 ROI 면적은  $708 \times 313 = 221,004\text{ px}$ 이다. 따라서 ROI 비율은 약 63.9%이며, 연산량은 약 36.1% 감소하는 것으로 계산된다.

$$R_{reduce} \approx 36.1\%$$

이는 실시간 추론 환경에서 FPS 향상과 탐지 안정성 확보에 기여하였다.



[그림 5] 고속도로 CCTV 영상에서의 ROI(Region of Interest) 지정 예시.

상단 원거리 구간은 배경에 해당하거나 픽셀 수 부족으로 탐지·추적 성능이 저하되어 제외하였으며, 차량 크기가 충분히 확보되는 하단 차로 영역만을 탐지 대상으로 설정하였다. ROI 기반 입력 축소는 배경 제거, 오탐률 감소, 탐지 속도 향상에 기여한다.

## 제 4절 인공지능 기반 차량 탐지 모듈 설계

본 연구에서는 고속도로 CCTV 환경에서 차량을 안정적으로 탐지하기 위해 인공지능 기반 객체 탐지 모델을 적용하였다. 여러 경량 모델 중에서 실시간성이 우수한 Ultralytics YOLOv11n을 최종적으로 채택하여 탐지 모듈을 구성하였다. YOLO 계열 모델은 단일 신경망 기반의 실시간 객체 탐지 방식으로 널리 활용되며, 그 중 YOLOv11n은 경량화된 구조를 통해 빠른 추론 속도를 제공하는 버전이다. 프레임당 객체 수가 많지 않고 차량 크기가 작은 고속도로 CCTV 환경에서는 이러한 경량 모델이 특히 적합하다.

전처리 단계에서는 카메라마다 설정된 ROI 영역과 해상도 정규화를 적용한 프레임을 모델 입력으로 사용한다. 모든 입력은 640×640 규격으로 변환되며, 원본 전체 프레임 대신 도로 영역만을 사용함으로써 탐지 안정성과 추론 속도를 동시에 향상시켰다. 실험에서도 ROI 적용 후 오탐률 감소와 FPS 향상 효과가 확인되었다.

탐지 결과는 각 객체에 대해 bounding box 좌표, 클래스, 그리고 신뢰도(confidence)를 포함한다. 여기서 confidence는 해당 영역에 실제 객체가 존재할 확률을 의미하며, “객체 존재 확률  $P(object)$ ”와 “가장 가능성이 높은 클래스 확률  $\max(P(class))$ ”의 곱으로 정의된다.

$$Confidence = P(object) \times \max(P(class))$$

또한 YOLO 계열 모델은 예측된 박스들의 중복 제거를 위해 IoU 기반의 NMS(Non-Maximum Suppression)을 사용하며, IoU는 예측 박스와 정답 박스의 겹침 비율을 의미한다.

본 연구에서는 COCO 사전학습 모델을 사용하고 자동차, 트럭, 버스 등 차량 관련 클래스만을 필터링하여 활용하였다. 고속도로 환경에서는 차량 이외의 객체가 거의 없기 때문에 추가적인 fine-tuning 없이도 실시간 처리 성능과 구현 복잡도 측면에서 효율적으로 운용할 수 있었다.

탐지된 bounding box는 이후 tracking 모듈로 전달되어 단일 카메라 내 ID 유지에 활용되며, 후속 카메라에서 동일 차량 후보를 식별하기 위한 Re-ID 모듈의 입력으로 사용된다.

## 제 5절 단일 카메라 기반 차량 추적 모듈 설계

본 연구의 단일 카메라 기반 차량 tracking 모듈은 탐지된 차량 객체를 프레임 단위로 연결하여 동일 객체의 연속적인 이동 궤적을 유지하도록 설계되었다. 이를 위해 Kalman filter 기반 상태 예측과 Hungarian 알고리즘 기반 매칭을 결합한 tracking-by-detection 구조를 사용하였다. 실제 구현에서는 Ultralytics YOLO가 제공하는 tracking 기능을 활용하여 본 구조를 효율적으로 적용하였다.

Kalman filter는 이전 프레임에서의 bounding box 위치와 속도 정보를 기반으로 현재 프레임에서의 객체 위치를 예측한다. 이를 통해 일시적인 탐지 누락(missing detection), 부분 가림(occlusion), 혹은 저해상도로 인한 detection 불안정 상황에서도 객체의 궤적이 끊기지 않도록 보완한다. 이후 매칭 단계에서는 예측 위치와 현재 프레임의 탐지 결과 간 IoU(Intersection over Union)를 기반으로 비용 행렬을 구성하고, Hungarian 알고리즘을 통해 전체 비용이 최소화되도록 탐지 결과와 기존 트랙을 연결한다.

또한 본 연구에서는 ByteTrack 계열의 tracking 방식에서 제안된 것처럼, 일정 신뢰도 이하의 탐지 결과도 매칭 후보로 활용하는 전략을 적용하였다. 고속도로 CCTV는 원거리 촬영 특성으로 인해 차량 크기가 매우 작아지고, 배경 노이즈가 많아 탐지 신호가 약해지는 경우가 빈번하다. 이러한 환경에서는 low-score detection을 활용하는 것이 추적 단절을 줄이고 ID 스위칭을 완화하는 데 효과적이다.

새로운 객체가 등장하면 새로운 local ID가 생성되며 tracking이 시작되고, 일정 시간 동안 탐지되지 않은 트랙은 자동으로 종료된다. 최종적으로 유지된 단일 카메라 내 tracking ID는 Re-ID 및 핸드오버 모듈에서 동일 차량 여부를 판단하는 핵심 입력으로 사용된다.

## 제 6절 색상 히스토그램 기반 차량 Re-ID 설계

고속도로 CCTV 환경에서는 차량이 원거리에서 촬영되기 때문에 외관의 세부 형태나 텍스처 정보를 충분히 확보하기 어렵다. 이로 인해 고해상도 입력을 필요로 하는 딥러닝 기반 Re-ID 모델은 실시간 처리 성능이 저하되거나 특징 품질이 불안정해지는 문제가 있다. 본 연구에서는 이러한 제약을 고려하여 연산량이 적고 조명 변화에 강인한 색상 기반 특징을 활용하는 경량 통계 기반 차량 Re-ID 방식을 설계하였다.

Re-ID 특징 추출은 단일 카메라 tracking 모듈에서 추적되고 있는 차량의 bounding box를 crop하는 단계에서 시작된다. crop된 차량 이미지 패치는 RGB 색상 공간보다 조명 변화에 강한 HSV(Hue, Saturation, Value) 색상 공간으로 변환되며, 특히 H(Hue)와 S(Saturation) 채널은 차량 외관의 색상 특성을 안정적으로 표현할 수 있다. 고속도로 CCTV는 날씨·시간·방향에 따라 전체 밝기 변화가 크게 나타나므로, HSV 기반 표현은 조도 변화로 인한 특징 왜곡을 줄이는데 효과적이다.

색상 정보를 보다 구조적으로 표현하기 위해 차량 이미지를 세로 방향으로 5개의 밴드(band)로 분할하여, 각 밴드에서 독립적으로 히스토그램을 계산하였다. 이는 차량 루프 - 유리창 - 차체 - 범퍼 - 휠 등 영역별 색상 분포가 상이하다는 점을 고려한 구조적 설계이다. 전체 이미지를 하나의 히스토그램으로 계산할 경우 지역적 색상 차이가 희석되지만, 밴드 단위 특징을 결합하면 외관 차이를 더 정밀하게 구분할 수 있다. 각 밴드의 히스토그램은 균일한 bin 수로 분할하여 계산하였으며, L1 정규화를 적용해 조도 변화와 이미지 크기 차이에 따른 영향을 완화하였다.

두 차량 간 유사도 비교는 Bhattacharyya 거리 기반 히스토그램 분포 비교 방



식을 사용하였다. 5개 밴드에서 계산된 거리 값은 가중 평균하여 최종 유사도 점수를 산출하며, 특정 임계값 이하일 경우 동일 차량으로 판단한다. 추출된 Re-ID 특징 벡터는 local ID와 함께 특징 DB에 저장되고, 후속 카메라에서 동일 차량 후보를 탐색하는 과정에서 활용된다.

제안된 색상 기반 Re-ID 방식은 학습 과정이 필요 없고 계산 복잡도가 낮아, 고해상도 추론이 어려운 고속도로 CCTV 환경과 실시간 다중 카메라 추적 시스템의 제약 조건에 적합한 경량 솔루션이라는 장점을 가진다.



[ 그림 6 ] 색상 기반 차량 Re-ID를 위한 밴드 분할 및 히스토그램 특징 추출 과정.

차량 크롭 이미지를 세로 방향으로 5개 밴드로 분할하고, 각 밴드에서 HSV 히스토그램을 계산하여 L1 정규화를 수행한다. 밴드별 지역적 색상 분포를 결합해 차량 외관의 차이를 정교하게 표현한다.

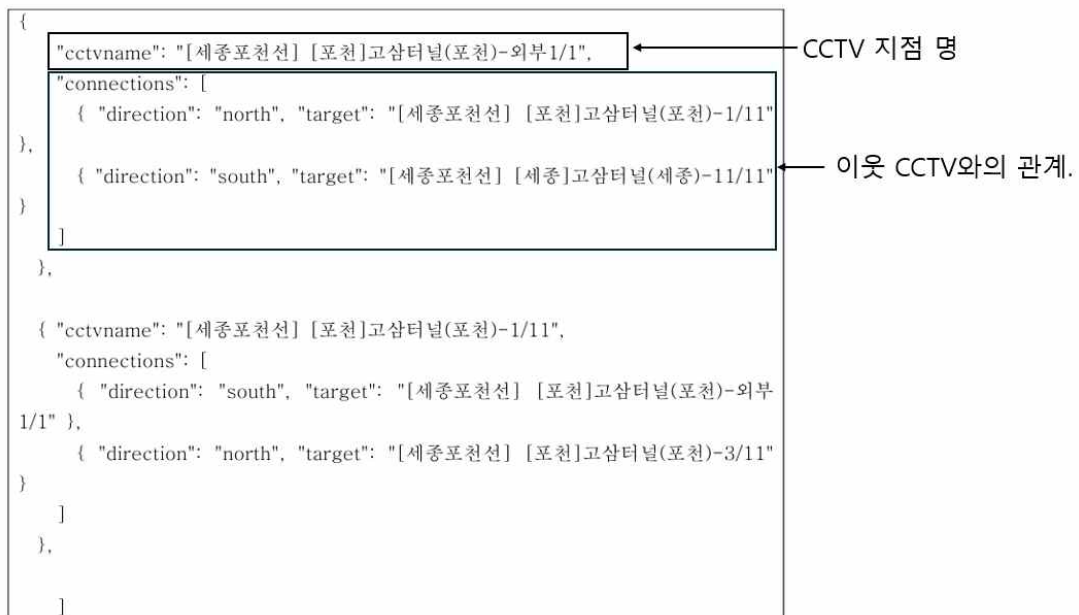
## 제 7절 다중 카메라 핸드오버 설계

다중 카메라 환경에서 동일 차량의 이동을 연속적으로 추적하기 위해서는 한 카메라에서 선택된 차량이 인접 카메라 화면에 등장했을 때 동일 객체인지 판별하는 과정이 필요하다. 초기 설계 단계에서는 객체가 화면 경계에 진입하거나 tracking 모듈이 객체를 lost 상태로 전환하는 시점을 이벤트로 활용하는 자동 핸드오버 방식을 고려하였다. 그러나 실제 고속도로 CCTV 영상은 해상도가 낮고 촬영 각도가 일정하지 않아 경계 진입 여부나 lost 이벤트를 안정적으로 검출하기 어려웠으며, 차량이 두 카메라 사이를 이동하는 시간 간격 또한 일정하지 않아 명확한 이벤트 조건을 정의하는 데 한계가 있었다.

이러한 문제를 해결하기 위해 본 연구에서는 이벤트 기반 방식 대신 사용자가 기준 차량을 선택한 이후 인접 카메라에서 동일 차량을 탐색하는 비이벤트 기반 연속 비교 방식으로 설계를 단순화하였다. 우선 시스템은 카메라 간 물리적 연결 관계를 표현한 CCTV Graph를 바탕으로, 기준 차량이 실제로 이동할 가능성이 있는 인접 카메라들을 탐색 후보로 한정한다. 사용자가 카메라를 전환하면 해당 카메라에서 탐지된 모든 차량에 대해 기준 차량의 색상 특징과의 유사도를 계산하여 가장 비슷한 객체를 후보로 제시한다.

유사도 계산은 HSV 색 공간 기반의 5개 밴드(band) 히스토그램을 활용하며, 각 밴드별 히스토그램 분포 간 Bhattacharyya 거리를 측정하여 두 차량 이미지 간의 색상 유사도를 정량화한다. 기준 차량 - 후속 카메라 후보 차량 간의 거리가 설정된 임계값 이하일 경우 동일 차량으로 판단하고, 초과할 경우 후보에서 제외한다. 이러한 방식은 딥러닝 기반 Re-ID 모델의 고해상도 의존성과 연산 부담을 피하면서도, 고속도로 CCTV의 원거리·저해상도 환경에서 안정적인 비교 성능을 제공한다는 장점이 있다.

제안된 핸드오버 구조는 차량의 화면 이탈 시점을 명확히 포착하지 않아도 되고, 프레임 기반 이벤트에 의존하지 않기 때문에 고속도로 CCTV 환경에 적합한 실용적인 설계 방식이다. 또한 사용자 선택 기반 구조로 동작하여 불확실한 이벤트 검출로 인한 오동작을 최소화하고, 인접 카메라의 모든 객체 중에서 기준 차량과 가장 유사한 후보를 지속적으로 탐색할 수 있어 다중 카메라 간 차량 경로 연계가 안정적으로 수행된다.



[ 그림 7 ] 고속도로 CCTV 간 물리적 연결 관계를 나타낸 CCTV Graph 구조.

각 CCTV는 고유 지점명(cctvname)과 인접 CCTV 목록(connections)으로 구성되며, 방향 정보(direction)와 대상(target)으로 연결된다. 이 구조는 차량 이동 가능 경로를 정의하여, 핸드오버 대상 카메라를 효율적으로 탐색하는 데 활용된다.

## 제 4장 시스템의 구현

### 제 1절 시스템 구현 환경

본 연구의 시스템은 실시간 스트리밍 처리와 딥러닝 기반 차량 탐지를 동시에 수행해야 하므로, 표 1과 같이 GPU 가속 환경과 최신 Python 기반 개발 환경을 구성하였다. 운영체제(OS, Operating System)는 Windows 10/11을 사용하였으며, 주요 라이브러리로는 Ultralytics YOLOv11과 OpenCV (Open Source Computer Vision Library)를 Python 3.11 환경에서 동작하도록 설정하였다.

실시간 탐지 속도를 확보하기 위해 NVIDIA RTX 3070Ti GPU와 Ryzen 7 5700X CPU를 사용하였으며, 입력 데이터는 ITS Open API (Open Application Programming Interface)를 통해 제공되는 고속도로 CCTV HLS (HTTP Live Streaming) 및 mp4 스트림을 사용하여 실제 관제 환경과 유사한 조건에서 시스템을 구현·검증하였다.

| 구분             | 내용                                                               |
|----------------|------------------------------------------------------------------|
| 1. 운영체제(OS)    | Windows 10 / 11 (64-bit)                                         |
| 2. Python 버전   | Python 3.11                                                      |
| 3. 사용 라이브러리    | Ultralytics YOLOv11, OpenCV, NumPy, Flask, Requests, Threading 등 |
| 4. 딥러닝 프레임워크   | PyTorch (Ultralytics YOLO 내장)                                    |
| 5. GPU         | NVIDIA RTX 3070ti                                                |
| 6. CPU         | AMD Rizen 7 5700x                                                |
| 7. RAM         | 32 GB                                                            |
| 8. IDE / 실행 환경 | VSCode, Conda(venv), Flask 개발 서버                                 |
| 9. 사용 데이터      | ITS OpenAPI 기반 실시간 고속도로 CCTV HLS/mp4 스트림                         |

[ 표 1 ] 개발 환경

## 제 2절 전체 파이프라인 구성 요약

본 시스템은 실시간 스트리밍 입력부터 차량 탐지·추적, Re-ID (Re-Identification), 다중 카메라 간 핸드오버(handover), 그리고 UI (User Interface) 시각화까지 하나의 파이프라인으로 구성되어 있으며, app.py를 중심으로 모든 모듈이 계층적으로 동작한다. 주요 기능은 스트림 관리, 프레임 전처리, 탐지·추적, 색상 기반 특징 추출, 핸드오버 제어, 사용자 조작으로 구분되며, 각 모듈은 독립적으로 작동하지만 전체 실행 흐름은 core.pipeline에서 통합적으로 관리한다.

| 단계                | 실행 파일/모듈                                                  | 주요 동작                                                            |
|-------------------|-----------------------------------------------------------|------------------------------------------------------------------|
| 1. 앱 실행           | app.py                                                    | Flask 앱 실행 및 PipelineState 노출. 실제 실행은 flask_template/app.py에 위임. |
| 2. Flask 초기화      | examples/flask_demo/flask_template/app.py                 | Flask 서버, SQLite DB, 로그인, Web UI 초기화.                            |
| 3. 파이프라인 백그라운드 실행 | start_pipeline_background()<br>core.pipeline.run_detect() | 파이프라인을 별도 스레드로 실행.<br>StreamManager/Tracker/ReID 초기화.            |
| 4. 스트림 입력         | core.stream_manager.HLSSStreamManager                     | HLS/MP4/RTSP URL 감지, OpenAPI로 URL 갱신, 프레임 리더 스레드 동작.             |
| 5. 카메라 전환         | core.switch_controller                                    | CCTV 그래프(JSON) 기반 다음 카메라 선택, a/d키 또는 클릭으로 스트림 재시작.               |
| 6. 탐지·추적          | detectors.yolo_tracker.<br>/yolotracker                   | ByteTrack 기반 Filtering, ROI 필터링, 차량 Local ID 생성.                 |
| 7. Re-ID 비교       | core.reid_bank,<br>core.pipeline_components.match_utils   | HSV 히스토그램 생성, 카메라별 화이트닝, 동일 차량 여부 검증.                            |
| 8. 핸드오버 기록        | core.pipeline_components.logging_utils                    | tracking_logs/에 차량 ID 변화와 핸드오버 이벤트 로그 저장.                        |
| 9. UI 전달          | /video_feed,<br>core.frame_bus.BUS                        | 최신 프레임 MJPEG 스트리밍, FPS 통계 갱신.                                    |
| 10. 사용자 명령        | /api/keypress,<br>/api/click→ PipelineState               | 웹 UI에서 발생한 키/클릭 이벤트가 파이프라인 큐로 전달되어 반영됨.                          |

[ 표 2 ] 엔트리 흐름 및 모듈 호출 관계 요약

### 제 3절 스트림 입력 및 전처리 구현

스트림 입력 단계는 HLS URL 갱신과 최신 프레임 유지 기능을 수행하며, 전처리는 ROI 크롭이 중심이다.

| 항목           | 내용                                      |
|--------------|-----------------------------------------|
| 1. 스트림 관리    | 최신 HLS/mp4 URL 갱신, 끊김 복구, 최신 프레임 유지 버퍼  |
| 2. 사용 모듈     | core/stream_manager.py, utils/stream.py |
| 3. 프레임 입력 방식 | 백그라운드 스레드로 지속 수신 → frame_bus로 전달        |
| 4. ROI 적용    | cam_stats.json 기반 도로 영역 크롭, 상단 원거리 제외   |
| 5. 전처리 목적    | 작은 차량 제거로 오탐/ID 스위칭 감소, YOLO 입력 효율화     |
| 6. 색상 공간 변환  | HSV 변환( Re-ID 특징 추출용 )                  |
| 7. 부가 기능     | crop_saver로 ROI 튜닝 시 샘플 저장              |

[ 표 3 ] 스트림 입력 및 전처리 구성 요약

## 제 4절 차량 탐지·추적 및 Re-ID 구현

탐지·추적은 YOLOv11n 기반 탐지기와 ByteTrack 계열의 YOLO Tracking 모듈을 사용하여 단일 카메라 내에서 안정적인 Local ID를 유지하도록 구현하였다. Re-ID는 저해상도 CCTV 특성을 고려하여 HSV 색상 기반 특징을 사용하고, 세로 5밴드 히스토그램과 Bhattacharyya 거리를 이용해 동일 차량 여부를 판단한다.

전체 구성은 [ 표 4 ]와 같다.

| 구성 요소          | 내용                                                          |
|----------------|-------------------------------------------------------------|
| 1. 탐지 모델       | Ultralytics YOLOv11n (COCO pretrained)                      |
| 2. 입력 프레임      | ROI 적용 후 640×640 resize                                     |
| 3. 주요 클래스      | car, bus, truck 등 COCO 차량 클래스                               |
| 4. 추적 방식       | YOLO Tracking (Kalman + Hungarian)                          |
| 5. 강화 로직       | ByteTrack 방식 low-confidence bbox 포함                         |
| 6. Local ID 관리 | 탐지 차량에 연속적 ID 부여, lost timeout 적용                           |
| 7. Re-ID 특징    | HSV + 세로 5밴드 히스토그램                                          |
| 8. 정규화         | L1 normalization                                            |
| 9. 거리 함수       | Bhattacharyya distance                                      |
| 10. 주요 파일      | yolo_tracker.py, feature_extractor.py, handover_features.py |
| 11. 특징 저장소     | reid_bank.py, reid_bank_ext.py                              |

[ 표 4 ] 탐지·추적·Re-ID 구성 요약

## 제 5절 다중 카메라 핸드오버 및 UI 구현

본 연구에서는 다중 카메라 간 동일 차량 식별을 위해 색상 기반 Re-ID를 사용하되, 고속도로 CCTV 환경에서 자동 핸드오버 방식이 불안정하다는 점을 고려하여 사용자 선택 기반 기준 차량 지정 방식을 채택하였다. 사용자가 특정 차량을 클릭하면 해당 차량의 crop 이미지가 HSV 색상 공간으로 변환되고, 다섯 개 세로 밴드 히스토그램으로 분할되어 Re-ID 기준 특징으로 저장된다. 이 특징은 카메라가 전환되어도 유지된다.

이후 시스템은 핸드오버 이벤트나 lost 여부를 판단하지 않고, 화면에 표시되는 인접 CCTV의 모든 차량을 대상으로 기준 특징과의 유사도를 매 프레임 계산한다. 인접 CCTV는 `cctv_graph_connections.json`에 정의된 상·하행 카메라 관계를 바탕으로 `SwitchController`를 통해 전환되며, 사용자는 UI 조작으로 다음 카메라로 자연스럽게 이동할 수 있다.

후속 카메라의 각 차량은 동일한 방식으로 HSV 변환 및 5밴드 히스토그램이 생성되며, Bhattacharyya 거리를 통해 기준 차량과의 유사도가 측정된다. 거리값이 작은 객체가 우선적으로 동일 차량 후보가 되며, UI상에서 bounding box 강조 및 로그 표시를 통해 실시간으로 비교 결과를 확인할 수 있다.

이 방식은 완전 자동화 방식 대비 구조가 단순하고 안정성이 높으며, 특히 저해상도·원거리 촬영이 많은 고속도로 CCTV 환경에서 유사 색 차량 간 오탐을 줄일 수 있다. 또한 기준 차량을 명시적으로 선택하는 구조는 디버깅과 Re-ID 튜닝에도 유리하여, Global ID 오염 문제 없이 다중 카메라 비교 기능을 제공한다.

UI는 Flask 기반으로 구현되며, 선택 차량의 Local ID, 후보 차량의 유사도, 그리고 실시간 스트리밍 영상이 함께 표시된다. 프레임 공유는 `frame_bus` 구조를

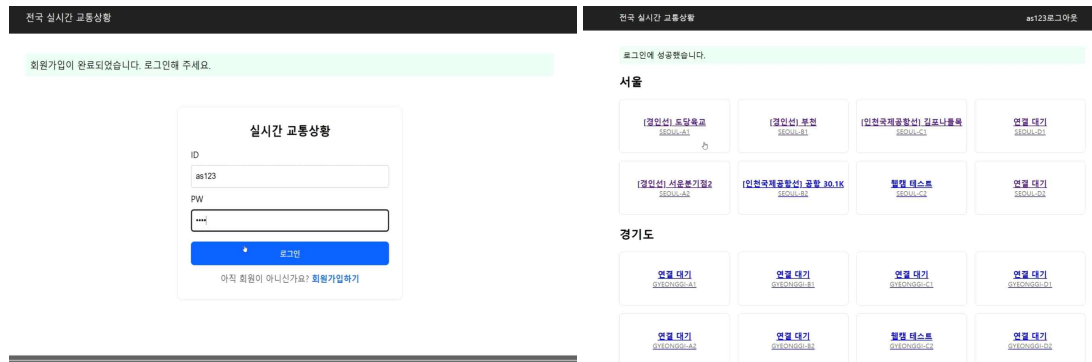


통해 이루어져 탐지·추적 결과와 Re-ID 비교 결과를 동시에 확인할 수 있다. 전체적으로 본 설계는 자동 핸드오버보다 단순하지만, 실제 환경에서 안정적으로 동작하는 실용적인 다중 카메라 비교 구조를 구성한다.

| 항목           | 내용                                                        |
|--------------|-----------------------------------------------------------|
| 1. 비교 기준     | 사용자가 클릭하여 지정한 기준 차량(Local ID)의 Re-ID 특징                   |
| 2. 비교 대상     | CCTV 그래프 기반 인접 카메라 화면에 존재하는 모든 차량                         |
| 3. 핸드오버 방식   | 자동 핸드오버 강제 X → 기준 차량 기반 실시간 거리 비교 방식                      |
| 4. 거리 계산 방식  | Bhattacharyya distance (HSV+5밴드 히스토그램)                    |
| 5. 후속 카메라 선정 | cctv_graph_connections.json 기반 상·하행 그래프                   |
| 6. 화면 전환 모듈  | switch_controller.py                                      |
| 7. Re-ID 기능  | feature_extractor.py, handover_features.py, reid_bank*.py |
| 8. UI 구성     | 실시간 영상 출력, ID 표시, 후보 유사도 표시, 클릭으로 기준 차량 설정                |
| 9. 이점        | 자동 핸드오버 오탐 방지, 사용자 중심의 차량 추적 검증 가능                        |

[ 표 5 ] 핸드오버 및 UI 구성 요약

## 제 6절 시스템 실행 화면



[ 그림 8 ] 실행화면 1  
(로그인 및 회원가입)

[ 그림 9 ] 실행화면 2  
(지점 선택)

사용자는 해당 페이지를 통해 원하는 지점 선택 가능.



[ 그림 10 ] 실시간 스트리밍 및 사용자 UI 구성.

상단은 실시간 추론 결과를 제공하는 스트리밍 영역이며, 중앙은 카메라 전환 및 추적 제어를 위한 사용자 조작 패널, 하단은 FPS·메모리 사용량 등 시스템 정보를 시각적으로 제공하는 영역이다.

## 제 5장 실험 및 성능 평가

### 제 1절 실험 환경 구성

본 연구의 다중 카메라 차량 추적 시스템은 실제 고속도로 CCTV 환경에서 동작하는 것을 목표로 설계되었다. 실험은 외부 환경 변화의 영향을 최소화하기 위해 조도 변화가 비교적 제한적인 고속도로 구간의 CCTV를 중심으로 수행하였다. 해당 구간은 영상 밝기·색상 편차가 안정적이어서, 본 연구에서 사용한 HSV(Hu, Saturatio, Value) 기반 색상 특징의 신뢰성을 평가하는 데 적합한 환경을 제공한다.

또한 특정 구간에서는 카메라 간 설치 간격이 일정하고 차량의 이동 방향이 명확하게 유지되므로, 인접 CCTV 간 차량 이동을 관찰하기 위한 실험 조건을 충족한다. 실험은 주간 시간대 중심으로 수행되었으며, 실시간 스트림 기반 파이프라인의 안정성과 처리 속도도 함께 평가하였다.

| 항목          | 내용                                             |
|-------------|------------------------------------------------|
| 1. 테스트 구간   | 조도 변화가 적은 고속도로 CCTV 3개 연속 구간                   |
| 2. 프레임 소스   | 국가교통정보센터 HLS/mp4 스트림                           |
| 3. 테스트 시간대  | 주간 중심, 일부 저조도 구간 포함                            |
| 4. 하드웨어     | RTX 3070ti, AMD Rizen 7 5700x, RAM 32GB        |
| 5. YOLO 모델  | YOLOv11n (COCO pretrained)                     |
| 6. Tracking | YOLO Tracking (Kalman + Hungarian + ByteTrack) |
| 7. Re-ID    | HSV 기반 5밴드 히스토그램 + Bhattacharyya 거리            |

[ 표 6 ] 실험 환경

## 제 2절 실험 결과.

본 절에서는 실제 고속도로 CCTV 스트림을 대상으로 수행한 실험을 통해 본 연구에서 제안한 차량 탐지·추적 파이프라인의 동작을 검증하였다. 실험은 ROI(Region of Interest) 기반 전처리, 단일 카메라 추적, 사용자 선택 기반 Re-ID(Re-Identification)의 세 가지 주요 기능을 중심으로 이루어졌으며, 시스템이 실시간 환경에서 안정적으로 차량을 연계 추적할 수 있는지를 확인하는 것을 목표로 하였다.

먼저 ROI 적용 실험에서는 원거리·저해상도 영역을 입력에서 제거함으로써, 작은 객체에서 발생하는 오탐(false positive) 감소, bounding box 안정성 증가, 추론 지연 감소 등의 효과를 얻을 수 있었다. 이는 고속도로 CCTV에서 발생하는 작은 객체 탐지 문제를 완화하는 데 유효한 전처리 전략임을 확인하였다.

단일 카메라 추적 실험에서는 YOLO(You Only Look Once) 기반 tracking 모듈을 활용하여, 한 카메라 내부에서 동일 차량의 Local ID(Identifier) 유지가 안정적으로 이루어짐을 확인하였다. 특히 차량이 부분적으로 가려지거나 일시적으로 탐지가 누락되는 경우에도 Kalman filter 기반 예측을 통해 궤적이 일정 수준 보존되었다.

Re-ID 실험에서는 사용자가 선택한 기준 차량의 HSV 히스토그램 기반 특징이 후속 카메라에서도 안정적으로 매칭되는지 평가하였다. 인접한 CCTV로 전환될 때마다 모든 후보 객체와 Bhattacharyya 거리 기반의 유사도 비교를 수행한 결과, 기준 차량과 가장 유사한 차량이 지속적으로 식별되며 다중 카메라 간 연속적인 차량 추적이 가능함을 확인하였다.

종합적으로, 본 연구의 파이프라인은 다음과 같은 점에서 실제 고속도로 CCTV 환경에서도 실효성 있게 동작함을 검증하였다.

ROI 기반 입력 축소를 통해 탐지 안정성과 효율성이 향상되었으며, YOLO tracking 기반 단일 카메라 추적에서 ID 유지가 안정적으로 이루어졌고, 사용자 선택 기반 Re-ID 비교를 통해 다중 카메라 간 차량 이동 연속성이 실제 환경에서도 높은 신뢰도로 유지되었다.

따라서 제안된 시스템은 고속도로 CCTV의 특성을 고려한 실시간 차량 추적 환경에서 충분한 적용 가능성을 갖는 것으로 판단된다.



[ 그림 11 ] ROI(Region of Interest) 적용 유무에 따른 탐지 결과 비교.

좌측은 ROI를 적용한 경우로, 원거리 영역이 제거되어 작은 차량의 오탐(false positive)이 감소하고 bounding box가 안정적으로 형성된다. 우측은 ROI 미적용 상태로, 화면 상단의 불필요한 영역까지 탐지 영역에 포함되어 검출 노이즈가 증가하는 모습을 확인할 수 있다.



[ 그림 12 ] 다중 카메라 간 동일 차량 추적 과정.

왼쪽 열은 첫 번째 카메라에서 기준 차량을 사용자가 선택하는 단계이며, 선택된 객체의 색상 히스토그램 특징이 생성된다. 가운데 열은 인접 카메라로 전환된 뒤 동일 차량을 Re-Identification(Re-ID) 비교를 통해 매칭한 결과로, 선택된 차량이 연속적으로 강조되어 표시된다. 오른쪽 열은 다음 카메라에서도 동일한 방식으로 기준 차량과의 유사도를 계산하여, 동일 차량이 지속적으로 식별됨을 보여준다. 각 행은 시간 순서에 따른 프레임을 나타낸다.

| 시간                      | 이벤트                                                                                                         | 카메라  | 차량 ID  | 신뢰도    | 메시지                                                                                                         |
|-------------------------|-------------------------------------------------------------------------------------------------------------|------|--------|--------|-------------------------------------------------------------------------------------------------------------|
| 2025-11-15 14:02:30.704 | MATCH                                                                                                       | 고삼터널 | ID:132 | MEDIUM | [동일 차량 발견] [세종포천선] [포천]고삼터널(포천)-3/11(ID:132) → [세종포천선] [포천]고삼터널(포천)-7/11(ID:154)   유사도: 0.640   신뢰도: MEDIUM |
| 2025-11-15 14:02:19.122 | MATCH                                                                                                       | 고삼터널 | ID:132 | HIGH   | [동일 차량 발견] [세종포천선] [포천]고삼터널(포천)-3/11(ID:132) → [세종포천선] [포천]고삼터널(포천)-5/11(ID:132)   유사도: 0.584   신뢰도: HIGH   |
| 2025-11-15 14:02:19.034 | MATCH                                                                                                       | 고삼터널 | ID:132 | HIGH   | [동일 차량 발견] [세종포천선] [포천]고삼터널(포천)-3/11(ID:132) → [세종포천선] [포천]고삼터널(포천)-5/11(ID:132)   유사도: 0.576   신뢰도: HIGH   |
| 2025-11-15 14:02:19.034 | [동일 차량 발견] [세종포천선] [포천]고삼터널(포천)-3/11(ID:132) → [세종포천선] [포천]고삼터널(포천)-5/11(ID:132)   유사도: 0.576   신뢰도: HIGH   |      |        |        |                                                                                                             |
| 2025-11-15 14:02:19.122 | [동일 차량 발견] [세종포천선] [포천]고삼터널(포천)-3/11(ID:132) → [세종포천선] [포천]고삼터널(포천)-5/11(ID:132)   유사도: 0.584   신뢰도: HIGH   |      |        |        |                                                                                                             |
| 2025-11-15 14:02:30.704 | [동일 차량 발견] [세종포천선] [포천]고삼터널(포천)-3/11(ID:132) → [세종포천선] [포천]고삼터널(포천)-7/11(ID:154)   유사도: 0.640   신뢰도: MEDIUM |      |        |        |                                                                                                             |

[ 그림 13 ] 차량 매칭 로그 및 로그 기반 UI 예시.

그림은 다중 카메라 환경에서 동일 차량 여부를 판단하기 위해 기록되는 매칭 로그와, 이를 사용자에게 시각적으로 보여주는 UI 화면을 나타낸다. 각 로그 항목에는 매칭 시점, 이벤트 유형, 해당 CCTV 위치, Local ID, 유사도(distance), 신뢰도(confidence), 그리고 매칭 판단의 근거 메시지가 함께 기록된다.

유사도(distance)는 선택된 기준 차량과 후보 차량 간의 HSV(Hue - Saturation - Value) 기반 5-밴드 히스토그램 분포를 비교하여 계산된 Bhattacharyya 평균 거리이며, 값이 0에 가까울수록 두 분포가 유사하고, 1에 가까울수록 서로 다르다. 시스템은 이 값을 기반으로 동일 차량 여부를 판단한다.

유사도(distance) 계산 공식

로그에 기록되는 distance 값은 아래 공식에 따라 계산된다.

(1) 5개 밴드 분할

crop된 차량 이미지  $I$ 를 세로로 5등분

$$I = \{I_0, I_1, I_2, I_3, I_4\}$$

(2) 각 밴드에서 HS 히스토그램 계산

$$h_i = Hist_{HS}(I_i)$$

(3) 각 밴드에서 Bhattacharyya 거리 계산

$$d_i = D_B(h_i^{(query)}, h_i^{(gallery)})$$

(4) 유효 밴드의 거리 평균

$$d_{mean} = \frac{1}{N} \sum_{i=0}^4 d_i$$

값이 0에 가까울수록 두 차량의 색 분포가 유사  
 값이 1에 가까울수록 다른 차량일 가능성이 높음  
 로그에 표시되는 distance는 바로 이  $d_{mean}$ 값이다.

#### 신뢰도 구간표

| 신뢰도    | 조건                          | 의미               |
|--------|-----------------------------|------------------|
| HIGH   | $d_{mean} < 0.60$           | 거의 동일한 색 분포      |
| MEDIUM | $0.60 \leq d_{mean} < 0.70$ | 허용 가능한 범위 내 유사   |
| LOW    | $0.70 \leq d_{mean} < 0.80$ | 애매한 구간, 추가 확인 필요 |
| REJECT | $d_{mean} \geq 0.80$        | 다른 차량으로 간주       |

[ 표 7 ] 신뢰도 구간표



### 제 3절 한계 및 고찰.

본 연구는 실제 고속도로 CCTV 스트림을 기반으로 다중 카메라 환경에서 차량을 연계 추적하는 시스템을 구현하는 것을 목표로 하였다. 연구 과정에서는 다양한 기능을 통합한 고도화된 파이프라인을 구상하였으나, 실제 교통 관제 CCTV의 환경적 제약과 기술적 한계로 인해 여러 차례 구조 수정이 이루어졌다. 본 절에서는 연구 과정에서 확인된 주요 한계를 정리하고, 설계 변경의 배경과 실험 결과에 대한 종합적 고찰을 제시한다.

연구 초기에는 다음과 같은 완전 자동화 Multi-Camera Tracking(MCT) 구조를 목표로 삼았다.

- 단일 카메라 탐지·추적 → 화면 경계 기반 이탈 이벤트 감지
- 딥러닝 기반 차량 Re-ID 네트워크 적용
- 카메라 간 이동 가능 경로 기반 후보 필터링
- 자동 동일 차량 판별 및 후속 카메라 핸드오버

이와 같은 자동 이벤트 기반 MCT 구조는 이론적으로는 유효하지만, 실제 고속도로 CCTV 환경에서는 다음과 같은 이유로 안정적 구현이 어려웠다.

차량 크기 자체가 지나치게 작음

- 10~25px 수준의 매우 작은 bounding box로 촬영됨
- Re-ID 모델이 활용하는 텍스처·세부 패턴 정보가 거의 존재하지 않음

해상도·조명 변화에 의한 외관 정보 손실

- 터널·야간·조명 반사 등으로 같은 차량이라도 색 분포가 달라질 수 있음

경계 기반 이탈 이벤트가 안정적으로 검출되지 않음

- 차량이 도로 중앙에서 갑자기 사라지는 경우(프레임 누락 등)
- ROI 밖에서 탐지가 누락되거나 오탐되는 경우
- 어느 지점이 “화면 이탈”인지 정의가 어려움

프레임 지연과 HLS 특성으로 인해 시간 동기화 불가

- 동일 카메라에서도 탐지 시점이 조금씩 흔들림
- 이벤트 기반 핸드오버의 전제인 “시점 일치”가 불가능함

이와 같은 문제로 초기 자동화된 핸드오버 구조는 현장 데이터 품질 현실을 고려할 때 실용성이 떨어지는 방식임을 확인하였다. 이러한 한계가 반복적으로 확인됨에 따라, 본 연구는 초기 목표였던 완전 자동화 방식 대신, 실제 환경 조건에 적합한 구조로 시스템을 단계적으로 재설계할 필요가 있었다. 이에 따라 이후 연구에서는 연산 구조, 추적 방식, Re-ID 전략을 포함한 전반적인 시스템 구조를 검토하고 수정하였다.

초기 시스템에서는 여러 카메라의 영상을 가로로 결합(concat)한 단일 입력 이미지에 대해 하나의 YOLO 모델로 객체 탐지를 수행한 후, 탐지된 bounding box를 카메라별 독립 추적기에 분배하는 구조를 설계하였다. 이 방식은 모델 수행 횟수를 최소화할 수 있어 연산 효율 측면에서 매우 우수했으며, 초기 목표였던 단일 탐지 모델 기반 다중 카메라 처리 구조를 실현할 수 있다는 장점이 있었다.

그러나 Kalman filter와 Hungarian 알고리즘 기반의 전통적 다중 객체 추적기는 탐지 결과가 들어온 이후 트랙이 생성되기까지 수 프레임의 초기 딜레이가

발생하였다. 고속도로 CCTV 환경에서는 차량이 화면에 머무르는 시간이 매우 짧고, 프레임 호출 간격이 일정하지 않기 때문에 이러한 초기 트랙 생성 지연은 곧 ID 스위칭과 추적 손실로 이어졌다.

특히 실험에서는 다음 현상이 반복적으로 관측되었다.

YOLO가 bounding box를 즉시 검출하더라도 트래커는 3~8프레임 이후에야 ID를 등록했고, 그 사이 차량이 화면을 벗어나거나 새로운 ID로 재등록되는 사례가 다수 발생하여 동일 차량임에도 연속 프레임 기반 MOT가 유지되지 못하는 문제가 발생하였다.

이 문제를 해결하기 위해 최종 구조는 카메라별로 YOLO 모델과 YOLO 내장 트래커를 독립적으로 실행하는 방식으로 전환되었다. YOLO 내장 트래커는 탐지 결과를 즉시 반영하여 ID를 부여하므로, 앞선 구조에서 발생한 초기 프레임 지연 문제가 제거되었으며 실제 도로 영상 조건에서 안정적으로 ID continuity를 유지할 수 있었다.

다만, 이와 같은 구조 변경은 연산량 부담을 증가시켰고, 결과적으로 초기 설계에 포함되었던 딥러닝 기반 Re-ID 네트워크를 실시간 처리 환경에서 적용하기 어려워졌다.

이 구조적 전환 이후, 실시간성을 유지하면서 카메라 간 동일 차량 여부를 판단하기 위한 대안으로 HSV 히스토그램 기반 Bhattacharyya distance 비교 방식을 적용하였다. 단일 카메라 상황에서는 이 방식이 동일 차량을 안정적으로 분류하였으며, 조명 변화에 대한 HSV 색상공간의 상대적 내성이 확인되었다.

그러나 다중 카메라 환경으로 확장하자 동일 차량임에도 색상 분포에 큰 차이가 발생하였으며, 원본 프레임 대비 HSV가 카메라별로 일관된 경향성을 보이지 않았다. 이는 다음 요인에 의해 발생한 것으로 분석된다.

- 각 CCTV의 자동 노출 및 화이트밸런스 차이
- 카메라 광학계 차이 (렌즈, 센서 DPI, ISO)
- 터널/야간 반사광, 역광, 습한 노면 상태
- ISP(이미지 신호 처리기)의 색 보정 알고리즘 차이

그 결과 단일 threshold 기반 동일 차량 판단은 불가능하였고, 실험적으로 카메라 쌍(camera pair)별 서로 다른 임계값을 설정하는 adaptive matching 전략이 필요하였다.

이는 색상 기반 Re-ID 방식이 실시간성과 구현 단순성 측면에서는 유효하나, 카메라 환경이 통일되지 않은 실제 교통 영상 조건에서는 절대 기준이 아닌 상대적 비교 방식으로만 의미를 가진다는 점을 확인한 실험 결과였다.

본 연구에서 확인된 실험적 결과는 단순히 탐지 모델의 성능 향상만으로는 다중 카메라 차량 추적 시스템이 안정적으로 동작하지 않는다는 점을 보여준다. 특히 추적기 설계는 탐지 모델 선택보다 오히려 시스템 구조와 더 밀접한 연관성을 가진다는 사실이 관찰되었다. 동일한 탐지 모델을 사용하더라도, 입력 영상 구성 방식, 트래커의 연동 구조, 탐지 결과가 반영되는 시점 등 파이프라인의 처리 구조에 따라 ID 지속성과 추적 안정성이 크게 달라졌으며, 이는 모델 성능이 아닌 시스템 아키텍처 관점에서의 설계가 실제 동작 신뢰도를 결정하는 핵심 요소임을 시사한다.

또한 Re-ID와 관련하여, 알고리즘의 정교함이나 네트워크 구조보다 먼저 고려

되어야 할 요소가 영상 데이터의 품질과 촬영 환경의 일관성이라는 점이 확인되었다. 즉, 차량 외형 기반 Re-ID는 모델 자체가 아무리 우수하더라도, 카메라 간 촬영 조건(밝기, 색 보정, 노출, 위치, 센서 특성 등)이 정렬되지 않으면 의미 있는 비교 기준을 구성하기 어렵다. 이러한 결과는 Re-ID가 단순히 분류 문제나 특징 추출 기술의 문제가 아니라, 데이터 수집 환경이 포함된 시스템 수준 문제라는 점을 강조한다.

더불어 본 연구는 실시간 시스템에서는 탐지 정확도가 절대적인 기준이 아니라는 점도 확인하였다. 특히 다중 카메라 기반 추적에서는 정확도보다 프레임 간 처리 지연이 존재하지 않는 연속적인 처리 과정, 즉 지연 없는 ID 유지가 우선적으로 요구된다. 연산 지연으로 인해 추적 ID가 늦게 등록되거나 전환되는 경우, 실제 차량 이동과 시스템 반응 사이에 시간적 괴리가 발생하며 이는 곧 MCT 시스템의 신뢰도를 저하시킨다. 따라서 본 연구는 실험실 환경에서의 정적 성능 수치보다, 실제 환경에서 연속성을 유지하며 안정적으로 동작하는지 여부가 더 중요한 판단 기준임을 확인하였다.

종합하면, 본 연구는 초기 목표였던 단일 기술 기반의 완전 자동화 MCT 구조보다는, 실제 교통 영상 환경과 처리 장비의 성능 조건이 반영된 실용적 설계 기반 시스템 구조가 필요하다는 결론에 도달하였다. 본 연구 과정에서 확보된 실험적 관찰은 향후 딥러닝 기반 Re-ID 적용을 위한 데이터 정렬, 카메라 보정, 해상도 개선, threshold 학습 기반 자동화, feature-level adaptation, homography 기반 보정 등 후속 연구 방향을 제시하는 기반으로 활용될 수 있다.

| 단계    | 설계 방향                | 핵심 기술                             | 한계               |
|-------|----------------------|-----------------------------------|------------------|
| 1. 초기 | 단일 YOLO Concat 처리    | Kalman + Hungarian                | ID 등록 지연, 실용성 낮음 |
| 2. 중간 | 구조 실험 및 Re-ID 적용     | HSV Histogram 비교                  | 카메라 간 색상 차이 문제   |
| 3. 최종 | 카메라별 독립 처리 및 실시간성 확보 | YOLO Tracker + Threshold Matching | 연산량 증가           |

[ 표 8 ] 시스템 구조 변화 및 설계 비교

## 제 6장 결론 및 향후 과제

### 제 1절 연구 결과 요약

본 연구에서는 고속도로 CCTV 환경에서 다중 카메라 간 차량을 연계하여 추적하기 위한 경량화 구조를 설계하고 구현하였다. 고속도로 CCTV는 촬영 거리가 멀고 차량 크기가 작아 탐지·추적 성능이 낮아지는 문제가 존재하므로, 본 연구는 ROI 기반 전처리를 통해 탐지 안정성을 확보하고, YOLO Tracking을 이용해 단일 카메라 내에서 차량 ID를 안정적으로 유지할 수 있는 구조를 제시하였다. 또한 대규모 학습 기반 Re-ID 모델 사용이 어려운 환경적 제약을 고려하여, 사용자가 선택한 기준 차량의 색상 히스토그램을 기반으로 인접 카메라에서 동일 차량을 비교하는 방식을 도입하였다.

실험 결과, ROI 적용은 백그라운드 간섭을 줄여 탐지 안정성을 향상시켰으며, YOLO Tracking은 단일 카메라 내에서 연속적인 ID 추적이 가능함을 확인하였다. 또한 사용자 선택 기반 색상 비교 방식은 인접 CCTV 화면에서도 기준 차량과 유사한 차량을 식별하는 데 실용적으로 활용될 수 있음을 보여주었다. 이를 통해 복잡한 Re-ID 모델이나 대규모 데이터 구축 없이도 다중 카메라 간 차량 연계 추적 기능의 구현 가능성을 확인하였다.

## 제 2절 연구의 한계

본 연구는 실시간 ITS CCTV 스트림을 대상으로 수행되었기 때문에 정답 데이터가 존재하지 않아 mAP, IDF1 등 정량적 지표를 기반으로 성능을 평가할 수 없었다. 이에 따라 실험은 정성적 분석에 의존하였으며, 동일 차량 여부 판단에서도 색상 기반 특징의 한계로 인해 조명 변화, 터널 구간의 반사, 차량 색상 유사도 등에 따라 유사도 값이 불안정하게 나타나는 경우가 있었다.

또한 본 시스템은 완전 자동 핸드오버 방식을 채택하지 않고, 사용자가 선택한 기준 차량을 중심으로 인접 카메라에서 유사도를 비교하는 구조로 구성되어 있다. 이는 저해상도 CCTV 환경에서는 실용적인 접근이지만, 자동 Global ID 유지 측면에서는 한계가 존재한다. 특히 카메라 간 화질·노출·각도 차이가 크기 때문에, 동일 차량이라도 영상 간 외형 특성이 일관되게 유지되지 않으며, 색상 기반 비교 방식만으로는 오탐/미탐 가능성을 완전히 제거하기 어렵다.

마지막으로, 본 시스템의 구조는 실시간성과 경량성을 우선하여 설계되었기 때문에 딥러닝 기반 Re-ID, 대규모 데이터 학습, 카메라 calibration 기반 보정과 같은 연산 집약적 방법은 적용되지 않았다. 이러한 설계 방향은 실제 환경에서의 즉시성과 구현 용이성 측면에서는 타당하나, 반대로 확장성·자동화 수준·범용성을 제한하는 요소로 작용하였다.

### 제 3절 향후 연구 과제

향후에는 단일 서버 기반 구조를 엣지 - 중앙 서버 형태로 확장하여, 각 CCTV 인근에서 탐지·추적 연산을 처리하고 특징 정보만 중앙으로 전달하는 분산 구조를 도입할 필요가 있다. 이를 통해 네트워크 부담을 줄이면서 다수의 CCTV를 동시에 처리할 수 있는 확장성을 확보할 수 있다.

또한 본 연구는 색상 기반 Re-ID를 사용하였으나 조명 변화나 해상도 한계로 인해 색상 정보가 일관적이지 않은 경우가 있었다. 이를 보완하기 위해 차량 윤곽·형태 같은 외형 특징을 결합한 경량 Re-ID 방식을 추가하면 비교 안정성을 향상시킬 수 있을 것이다.

마지막으로 Tracking 과정에서 얻을 수 있는 차량의 이동 속도와 위치 변화를 활용하면, 차량의 다음 카메라 도착 위치 예측, 특정 차량의 이동 경로 분석, 검문소·관문 위치 선정 등 실무적 활용으로 확장할 가능성이 있다.



## 참 고 문 헌

- [1] J. Redmon et al., “You Only Look Once: Unified, Real-Time Object Detection,” CVPR, 2016.
- [2] J. Redmon and A. Farhadi, “YOLO9000: Better, Faster, Stronger,” CVPR, 2017.
- [3] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” arXiv, 2018.
- [4] C.-Y. Wang et al., “YOLOv7: Trainable Bag-of-Freebies,” arXiv, 2022.
- [5] Ultralytics, “YOLOv8 Technical Report,” 2023.
- [6] W. Liu et al., “SSD: Single Shot Multibox Detector,” ECCV, 2016.
- [7] 고준형, “차량 객체 탐지를 위한 딥러닝 모델 성능 비교 분석,” 석사학위논문, 경기대학교, 2022.
- [8] M. Tan et al., “EfficientDet: Scalable and Efficient Object Detection,” CVPR, 2020.
- [9] 박상진 외, “CCTV 영상을 활용한 동적 객체의 위치 추적 및 시각화 방안,” 지적과 국토정보, vol. 51, no. 1, pp. 53-65, 2021.
- [10] R. E. Kalman, “A New Approach to Linear Filtering and Prediction Problems,” Transactions of the ASME, 1960.
- [11] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” Naval Research Logistics Quarterly, 1955.
- [12] Y. Zhang et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” ECCV, 2022.
- [13] Y. Zhang et al., “FairMOT: Fair Multi-Object Tracking,” ECCV, 2020.
- [14] X. Zhou et al., “CenterTrack: Tracking Objects as Points,” ECCV, 2020.
- [15] Ultralytics, “YOLOv8 Tracking Documentation,” 2023.
- [16] X. Liu et al., “A Deep Learning-Based Approach to Vehicle Re-ID on the VeRi-776 Dataset,” ECCV Workshops, 2016.

- [17] J. He et al., “Region-Aware Deep Feature for Vehicle Re-Identification,” CVPR, 2019.
- [18] Y. Shen et al., “Learning Deep Neural Networks for Vehicle Re-Identification with Coupled Structure,” TIP, 2017.
- [19] S. Tangkawanit and S. Kanprachar,  
 “Performance of HSV Color System in Vehicle Detection Application under Different Illumination Intensities,” 2020.
- [20] 권용욱 외,  
 “스마트 차량 관리 시스템을 위한 HSV 색상모델 기반 키 프레임 추출 기법,”  
 한국전자통신학회 논문지, 2013.
- [21] 송두현 외, “패턴매칭과 HSV 컬러모델을 병용한 차량 인식 알고리즘,”  
 한국멀티미디어학회 학술발표논문집, 2004.
- [22] Y. Chen et al., “Multi-Camera Vehicle Re-Identification via Visual Embedding,” CVPR, 2018.
- [23] X. Liu et al., “A Deep Learning-Based Approach to Vehicle Re-ID on the VeRi-776 Dataset,” ECCV Workshops, 2016.
- [24] S. Tang et al., “Single-Shot Multi-Camera Tracking by Graph Reinforcement Learning,” ECCV, 2018.
- [25] K. Kim et al., “Multi-Camera Vehicle Tracking with Spatio-Temporal Constraints,” IEEE TITS, 2017.
- [26] Zhao et al., “Lightweight Vehicle Re-ID Using Color Histogram and Local Features,” IEEE TITS, 2020.
- [27] 장준혁 외, “Homography Transform을 활용한 딥러닝 기반 다중 카메라 차량 재인식 기술,” 한국전자기술연구원, 2023.
- [28] 방상성, “다중카메라를 이용한 광범위 영역의 객체 추적에 관한 연구,” 석사학위논문, 목

포대학교, 2015.